EJERCICIO. CLASES Y OBJETOS. POLIMORFISMO

Solución al ejercicio — Polimorfismo con Animales

Clase base Animal + clases derivadas Perro, Gato y Pato — Proyecto de Consola (.NET)

Objetivo del ejercicio:
Demostrar POLIMORFISMO: llamar al mismo método (EmitirSonido) sobre una referencia del tipo base (Animal) y obtener comportamientos distintos según el tipo real del objeto (Perro/Gato/Pato).

1) Qué vas a crear

- Un proyecto de consola en C# (Console App).
- Una clase base: Animal (con 3 propiedades).
- Tres clases derivadas: Perro, Gato y Pato (cada una con 2 propiedades propias).
- Un método EmitirSonido() que se redefine en cada animal para mostrar polimorfismo.
- Un Program.cs que crea una lista List<Animal> y recorre llamando a EmitirSonido().

2) Pasos (muy rápidos)

- 1) Abre Visual Studio
- 2) Crea un proyecto Console App.
- 3) Añade 4 archivos: Animal.cs, Perro.cs, Gato.cs, Pato.cs (o pega todo en Program.cs).
- 4) Copia y pega el código de este documento.
- 5) Ejecuta y observa el resultado en consola.

3) Código

3.1) Animal.cs (clase base)

```
using System;

namespace PolimorfismoAnimales
{
    // Clase BASE (padre)
    public class Animal
    {
        // Propiedades comunes a todos los animales
        public string Nombre { get; set; }
        public int Edad { get; set; }
        public string Color { get; set; }

        public Animal(string nombre, int edad, string color)
        {
            Nombre = nombre;
            Edad = edad;
            Color = color;
        }

        // virtual => permite que las clases hijas lo redefinan (override)
        public virtual void EmitirSonido()
        {
            Console.WriteLine("Sonido genérico de animal...");
```

```
        }

        public void MostrarFicha()
        {
            Console.WriteLine($"Animal: {Nombre} | Edad: {Edad} | Color: {Color}");
        }
    }
}
```

3.2) Perro.cs (clase derivada)

```
using System;

namespace PolimorfismoAnimales
{
    public class Perro : Animal
    {
        // Propiedades propias del perro (2)
        public string Raza { get; set; }
        public bool EsGuardian { get; set; }

        public Perro(string nombre, int edad, string color, string raza, bool esGuardian)
            : base(nombre, edad, color) // llama al constructor del padre
        {
            Raza = raza;
            EsGuardian = esGuardian;
        }

        public override void EmitirSonido()
        {
            Console.WriteLine("guau");
        }
    }
}
```

3.3) Gato.cs (clase derivada)

```
using System;

namespace PolimorfismoAnimales
{
    public class Gato : Animal
    {
        // Propiedades propias del gato (2)
        public bool TieneBigotes { get; set; }
        public int Vidas { get; set; }

        public Gato(string nombre, int edad, string color, bool tieneBigotes, int vidas)
            : base(nombre, edad, color)
        {
            TieneBigotes = tieneBigotes;
            Vidas = vidas;
        }

        public override void EmitirSonido()
        {
            Console.WriteLine("miau");
        }
    }
}
```

3.4) Pato.cs (clase derivada)

```
using System;

namespace PolimorfismoAnimales
{
    public class Pato : Animal
    {
        // Propiedades propias del pato (2)
        public bool PuedeVolar { get; set; }
        public string TipoPico { get; set; }

        public Pato(string nombre, int edad, string color, bool puedeVolar, string tipoPico)
            : base(nombre, edad, color)
        {
            PuedeVolar = puedeVolar;
            TipoPico = tipoPico;
        }

        public override void EmitirSonido()
        {
            Console.WriteLine("cua cua");
        }
    }
}
```

3.5) Program.cs (donde se ve el polimorfismo)

```
using System;
using System.Collections.Generic;

namespace PolimorfismoAnimales
{
    class Program
    {
        static void Main()
        {
            Console.WriteLine("=== EJEMPLO DE POLIMORFISMO EN C# ===\n");

            // Creamos animales concretos (clases hijas)
            Animal a1 = new Perro("Rocky", 3, "Marrón", "Pastor Alemán", true);
            Animal a2 = new Gato("Misu", 2, "Blanco", true, 7);
            Animal a3 = new Pato("Donald", 1, "Amarillo", true, "Ancho");

            // Lista del tipo BASE: Animal
            List<Animal> animales = new List<Animal> { a1, a2, a3 };

            Console.WriteLine("Recorremos la lista y llamamos al MISMO método (EmitirSonido):\n");

            foreach (Animal animal in animales)
            {
                animal.MostrarFicha();

                // POLIMORFISMO:
                // Aunque la variable sea Animal, se ejecuta la versión override
                // del tipo real (Perro/Gato/Pato).
                animal.EmitirSonido();

                Console.WriteLine("-------------------------");
            }

            Console.WriteLine("\nFin. Pulsa una tecla para salir...");
            Console.ReadKey();
        }
    }
}
```

4) Explicación

- Animal tiene EmitirSonido() marcado como virtual: “esto se puede redefinir en los hijos”.
- Perro/Gato/Pato lo redefinen con override y ponen su sonido.
- En Program, guardamos todos en una List<Animal>.
- Llamamos a EmitirSonido() sobre Animal, pero cada objeto responde distinto: eso es polimorfismo.

5) Propuesta profe

- Crea una clase Vaca : Animal con 2 propiedades propias y sonido "muu".
- Añádela a la lista y comprueba que el foreach funciona sin cambiar nada.